Functions

Introduction

- In c, we can divide a large program into the basic building blocks known as function.
- The function contains the set of programming statements enclosed by {}.
- A function can be called multiple times to provide reusability and modularity to the C program. In other words, we can say that the collection of functions creates a program.

Definition

- A function is a group of statements that together perform a task.
- Every C program has at least one function, which is **main()**, and all the most trivial programs can define additional functions.

Types of function

There are two types of function in C programming:

- Standard library functions
- User-defined functions

Standard library functions

The standard library functions are built-in functions in C programming.

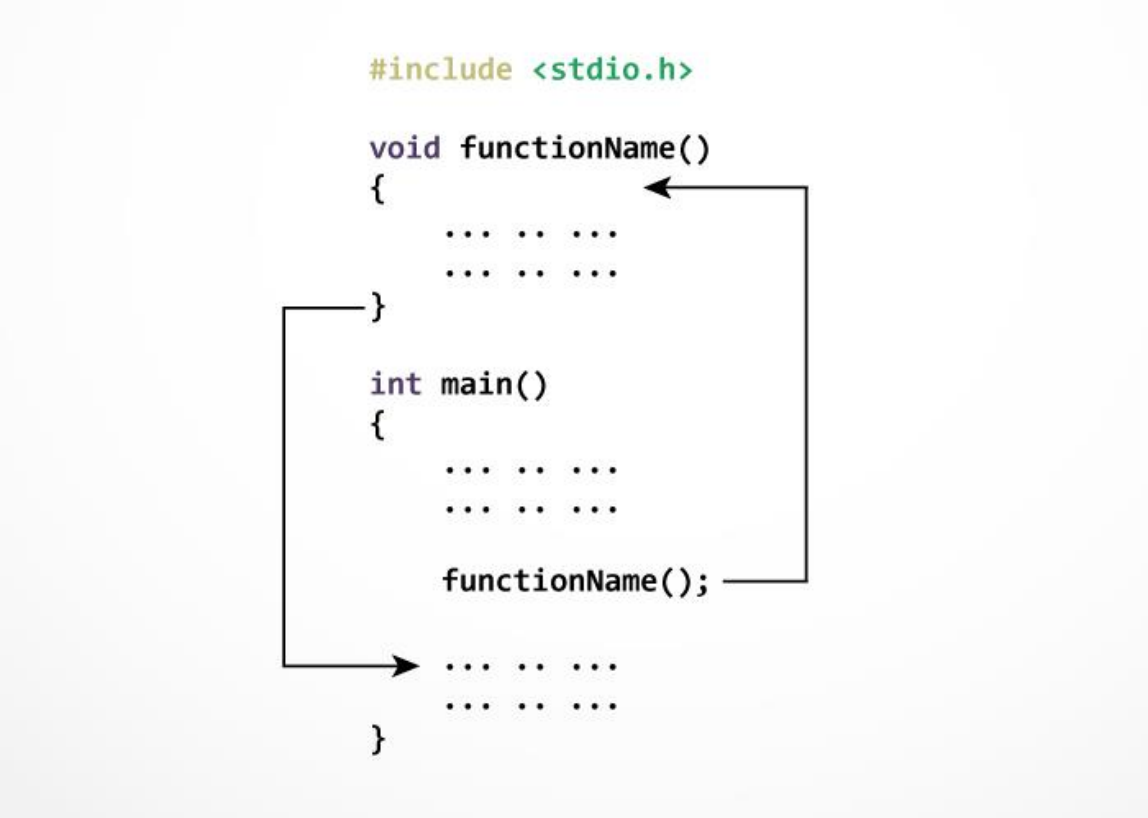
These functions are defined in header files. For example,

- The printf() is a standard library function to send formatted output to the screen . This function is defined in the stdio.h header file.
  Hence, to use the printf() function, we need to include the stdio.h header file using #include <stdio.h>.
- The sqrt() function calculates the square root of a number. The function is defined in the math.h header file.

User-defined function

You can also create functions as per your need. Such functions created by the user are known as user-defined functions.

How user-defined function works?



- The execution of a C program begins from the main() function.
- When the compiler encounters functionName();, control of the program jumps to void functionName() And, the compiler starts executing the codes inside functionName().
- The control of the program jumps back to the main() function once code inside the function definition is executed.

Function Aspects

- Function declaration
- Function call
- Function definition

Function Declarations

- A function **declaration** tells the compiler about a function's name, return type, parameters and how to call the function. The actual body of the function can be defined separately.
- A function can also be referred as a method or a sub-routine or a procedure, etc.

A function declaration has the following parts −

```
return_type function_name( parameter list );
```

Example

```
int max(int num1, int num2);
```

Parameter names are not important in function declaration only their type is required, so the following is also a valid declaration −

```
int max(int, int);
```

Defining a Function

The general form of a function definition in C programming language is as follows −

```
return_type function_name( parameter list )
{
   body of the function
}
```

A function definition in C programming consists of a *function header* and a *function body*. Here are all the parts of a function −

- **Return Type** − A function may return a value. The **return_type** is the data type of the value the function returns. Some functions perform the desired operations without returning a value. In this case, the return_type is the keyword **void**.
- **Function Name** − This is the actual name of the function.
- **Parameters** − A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. This value is referred to as actual parameter or argument.
- **Function Body** − The function body contains a collection of statements that define what the function does.

Example

Given below is the source code for a function called **max()**. This function takes two parameters num1 and num2 and returns the maximum value between the two −

```
/* function returning the max between two numbers */
int max(int num1, int num2) {

   /* local variable declaration */
   int result;

   if (num1 > num2)
      result = num1;
   else
      result = num2;
```

```
  return result;
}
```

Calling a Function

When a program calls a function, the program control is transferred to the called function.

A called function performs a defined task and when its return statement is executed or when its function-ending closing brace is reached, it returns the program control back to the main program.

To call a function, you simply need to pass the required parameters along with the function name, and if the function returns a value, then you can store the returned value.

Return Value

A C function may or may not return a value from the function. If you don't have to return any value from the function, use void for the return type.

Let's see a simple example of C function that doesn't return any value from the function.

Example without return value:

```
void hello()
{
printf("hello c");
}
```

If you want to return any value from the function, you need to use any data type such as int, long, char, etc. The return type depends on the value to be returned from the function.

Let's see a simple example of C function that returns int value from the function.

Example with return value:

```
int get()
{
return 10;
}
```

In the above example, we have to return 10 as a value, so the return type is int. If you want to return floating-point value (e.g., 10.2, 3.1, 54.5, etc), you need to use float as the return type of the method.

```
float get()
{
return 10.2;
}
```

Different aspects of function calling

A function may or may not accept any argument. It may or may not return any value. Based on these facts, There are four different aspects of function calls.

- function without arguments and without return value
- function without arguments and with return value
- function with arguments and without return value
- function with arguments and with return value

Actual parameters: The parameters that appear in function calls.
Formal parameters: The parameters that appear in function declarations.

Example for Function without argument and without return value

Example 1: program to add two number

```
#include <stdio.h>
void sum();

void main()
{
   printf("\nGoing to calculate the sum of two numbers:\n");
   sum();
}
void sum()
{
   int a,b;
   printf("\nEnter two numbers\n");
   scanf("%d %d",&a,&b);
   printf("The sum is %d",a+b);

}
```

Output

```
Going to calculate the sum of two numbers:

Enter two numbers 10
24
```

The sum is 34

Example 2: program to find maximum of two number

```
#include <stdio.h>
void max();

int main ()
{
  printf("Maxmimum of two number is\t") ;
  max();

  return 0;
}

void max()
{

 int a = 100;
  int b = 200;
  int result;

  if (a > b)
    result = a;
  else
    result = b;

  printf("%d",result);
}
```

Output

Maximum of two number is 200

Example 3: program to calculate the area of the square

```
#include<stdio.h>

void square();

void main()
{
   printf("Going to calculate the area of the square\n");
   square();

}
void square()
{
   int A,side;
   printf("Enter the length of the side : ");
```

```
    scanf("%d",&side);
    A= side * side;
   printf("The area of the square: %d\n",A);

}
```

Output

```
Going to calculate the area of the square
Enter the length of the side :  1010
The area of the square: 100
```

Example for Function without argument and with return value

Example 1: program to add two number

```
#include<stdio.h>
int sum();
void main()
{
   int result;
   printf("\nGoing to calculate the sum of two numbers:");
   result = sum();
   printf("The sum is %d",result);
}
int sum()
{
   int a,b;
   printf("\nEnter two numbers");
   scanf("%d %d",&a,&b);
   return a+b;
}
```

Output

```
Going to calculate the sum of two numbers:

Enter two numbers 10
24

The sum is 34
```

Example 2: program to find maximum of two number

```
#include <stdio.h>
int max();

int main ()
```

```
{
    int r;
   printf("Maxmimum of two number is\t") ;
   r=max();
printf("%d",r);
   return 0;
}

int max()
{

 int a = 100;
   int b = 200;
   int result;

   if (a > b)
      result = a;
   else
      result = b;

   return result ;
}
```

Output

Maximum of two number is 200

Example 3: program to calculate the area of the square

```
#include<stdio.h>

int square();

void main()
{
    int ret;
    printf("Going to calculate the area of the square\n");

   ret=square();
  printf("The area of the square: %d\n",ret);

}
int square()
{
    int A, side;
printf("Enter side\t");
    scanf("%d",&side);
    A= side * side;
   return A;
}
```

Output

Going to calculate the area of the square
Enter the length of the side : 1010
The area of the square: 100

Example for Function with argument and without return value

Example 1: program to add two number

```
#include<stdio.h>

void sum(int, int);

void main()
{
   int a,b,result;
   printf("\nGoing to calculate the sum of two numbers:\n");
   printf("\nEnter two numbers:\n");
   scanf("%d %d",&a,&b);
   sum(a,b);
}
void sum(int a, int b)
{
   printf("\nThe sum is %d",a+b);

}
```

Output

Going to calculate the sum of two numbers:

Enter two numbers 10
24

The sum is 34

Example 2: program to find maximum of two number

```
#include <stdio.h>
void max(int, int);

int main ()
{
    int a = 100,b = 200;
  printf("Maxmimum of two number is\t") ;
 max(a, b);
  return 0;
}
```

```
void max(int a, int b)
{


  int result;

  if (a > b)
    result = a;
  else
    result = b;

  printf("%d",result) ;
}
```

Output

Maximum of two number is 200

Example 3: program to calculate the area of the square

```
#include<stdio.h>

void square(int);

void main()
{
   int side;
   printf("Going to calculate the area of the square\n");
   printf("Enter side\t");
   scanf("%d",&side);
   square(side);

}
void square(int side)
{
   int A;
   A= side * side;
  printf("The area of the square: %d\n",A);
}
```

Output

Going to calculate the area of the square
Enter the length of the side : 1010
The area of the square: 100

Example for Function with argument and with return value

Example 1: program to add two number

```
#include<stdio.h>

int sum(int, int);

void main()
{
   int a,b,result;
   printf("\nGoing to calculate the sum of two numbers:\n");
   printf("\nEnter two numbers:\n");
   scanf("%d %d",&a,&b);
   result = sum(a,b);
   printf("\nThe sum is : %d",result);
}
int sum(int a, int b)
{
   return a+b;
}
```

Output

```
Going to calculate the sum of two numbers:
Enter two numbers:10
20
The sum is : 30
```

Example 2: program to find maximum of two number

```
#include <stdio.h>
int max(int, int);

int main ()
{
    int r, a = 100,b = 200;
  printf("Maxmimum of two number is\t") ;
 r=max(a, b);
  printf("%d",r) ;
  return 0;
}

int max(int a, int b)
{


  int result;

  if (a > b)
     result = a;
```

```
  else
    result = b;

 return result;
}
```

Output

Maximum of two number is 200

Example 3: program to calculate the area of the square

```
#include<stdio.h>

int square(int);

void main()
{
    int side, ret;
    printf("Going to calculate the area of the square\n");
    printf("Enter side\t");
    scanf("%d",&side);
   ret=square(side);
  printf("The area of the square: %d\n",ret);

}
int square(int side)
{
    int A;
    A= side * side;
   return A;
}
```

Output

Going to calculate the area of the square
Enter the length of the side : 1010
The area of the square: 100